

Relazione generale del progetto — Building AI Agent

Introduzione

Il progetto **Building AI Agent** nasce con l'obiettivo di costruire un agente AI partendo da zero e facendolo evolvere passo dopo passo.

L'idea principale non è soltanto creare un notebook che funziona, ma imparare come progettare un agente in modo ordinato, modulare e professionale.

Ogni versione introduce un solo concetto nuovo.

Questo permette di capire bene ogni componente prima di passare al successivo.

Il progetto segue una crescita naturale:

```
da agente semplice  
a sistema modulare  
a agente avanzato  
a struttura professionale
```

Obiettivo del progetto

L'obiettivo è costruire un agente capace di:

- ricevere una richiesta dell'utente;
- capire quale azione deve eseguire;
- scegliere il tool corretto;
- estrarre gli argomenti necessari;
- validare i dati;
- eseguire il tool;
- salvare lo stato dell'interazione;
- evolvere verso una struttura sempre più professionale.

Il progetto serve anche per imparare i concetti fondamentali degli agenti AI moderni:

- routing;
- parsing;
- execution;
- memory;
- planning;
- tool registry;
- uso controllato delle LLM;
- multi-step reasoning;
- organizzazione modulare del codice.

Filosofia del progetto

La filosofia del progetto è:

una versione, un concetto

Ogni versione aggiunge una sola evoluzione importante.

Questo approccio evita di correre troppo e permette di capire davvero perché ogni componente viene introdotto.

Non costruiamo subito un agente complesso.

Prima creiamo una base semplice.

Poi separiamo le responsabilità.

Poi rendiamo il codice più generico.

Poi introduciamo componenti più avanzati.

Questa è una mentalità da progetto professionale.

Architettura generale dell'agente

L'agente viene costruito come una pipeline di componenti.

Il flusso base è:

User
↓
Router
↓
Parser
↓
Executor
↓
Memory

Ogni componente ha una responsabilità precisa.

Il **Router** sceglie il tool.

Il **Parser** estrae gli argomenti.

L'**Executor** valida ed esegue.

L'**Agent** coordina il flusso.

La **Memory** salva lo stato.

Questa separazione rende il codice più chiaro, più estendibile e più facile da mantenere.

Router

Il Router ha il compito di capire quale tool usare.

Quando l'utente scrive una richiesta, il Router analizza il testo e sceglie l'azione più adatta.

Esempio:

```
What is 2 + 3?
```

Il Router deve capire che il tool corretto è:

```
"addition"
```

In una prima fase il Router può funzionare con regole e sinonimi.

In una fase più avanzata potrà essere migliorato con una LLM.

Il suo obiettivo è rispondere alla domanda:

```
Quale tool serve per questa richiesta?
```

Parser

Il Parser ha il compito di trasformare la richiesta dell'utente in argomenti utilizzabili dai tool.

Esempio:

```
What is 2 + 3?
```

diventa:

```
{"a": 2, "b": 3}
```

Oppure:

```
Hello, my name is luca.
```

diventa:

```
{"name": "luca"}
```

Il Parser è fondamentale perché collega il linguaggio naturale dell'utente ai parametri delle funzioni Python.

La sua evoluzione naturale è passare da un parser rigido a un parser generico, capace di leggere i parametri richiesti dal tool e usare extractor dedicati.

Executor

L'Executor ha il compito di eseguire il tool scelto.

Riceve:

```
tool_name  
arguments
```

Poi controlla che gli argomenti siano validi ed esegue la funzione corretta.

La sua responsabilità è separata da quella dell'Agent.

La regola è:

```
Agent = coordina  
Executor = valida ed esegue
```

Questa separazione è importante perché evita che l'Agent diventi troppo grande e difficile da mantenere.

Agent

L'Agent è il coordinatore principale del sistema.

Il suo compito non è fare tutto direttamente.
Il suo compito è orchestrare i vari componenti.

Il flusso logico è:

```
riceve la richiesta  
↓  
attiva il router  
↓  
chiama il parser
```

```
↓  
passa i dati all'executor  
↓  
salva lo stato in memoria
```

In una struttura professionale, l'Agent deve rimanere pulito.

Non deve contenere troppa logica specifica.
Deve coordinare componenti specializzati.

Memory

La Memory salva le interazioni dell'agente.

Ogni richiesta produce uno stato simile a questo:

```
{  
  "request": "What is 2 + 3?",  
  "action": "addition",  
  "arguments": {"a": 2, "b": 3},  
  "result": 5  
}
```

La memoria permette di tenere traccia di:

- richiesta ricevuta;
- tool scelto;
- argomenti estratti;
- risultato prodotto.

All'inizio può essere una semplice lista.

Successivamente può diventare un componente dedicato, con funzioni per salvare, leggere e cancellare lo stato.

Planner

Il Planner introduce un livello più avanzato.

Il suo compito è creare un piccolo piano prima dell'esecuzione.

Un piano può essere:

```
{  
  "tool": "addition",  
  "reason": "Add two numbers.",  
}
```

```
  "needs_arguments": ["a", "b"]  
}
```

Il Planner permette all'agente di ragionare prima di agire.

Non si limita a eseguire subito.

Prima crea una rappresentazione chiara di cosa deve fare.

Questo è un passaggio importante verso agenti più evoluti e multi-step.

Tool Registry

Il Tool Registry serve a gestire i tool in modo più ordinato.

All'inizio i tool possono stare dentro un dizionario scritto manualmente.

Con il tempo, però, è meglio registrarli con una funzione dedicata:

```
register_tool(...)
```

Il Tool Registry permette di:

- aggiungere nuovi tool più facilmente;
- mantenere i metadata ordinati;
- evitare dizionari troppo lunghi;
- rendere il sistema più estendibile.

La regola è:

```
i tool non devono solo esistere  
devono essere registrati e descritti bene
```

LLM Router

In una fase più avanzata, una LLM può essere usata per scegliere il tool migliore.

Il Router basato su regole funziona bene per richieste semplici, ma può essere limitato.

Per esempio:

```
Can you calculate the total of five and seven?
```

Un Router semplice potrebbe non riconoscere subito l'intento.

Una LLM, invece, può capire che l'utente vuole fare una somma e scegliere:

```
"addition"
```

La LLM deve essere introdotta solo dopo aver costruito una buona struttura classica.

Non deve essere usata come scorciatoia.
Deve migliorare un sistema già ordinato.

LLM Parser

La LLM può essere usata anche per estrarre argomenti.

Un parser a regole può riconoscere facilmente:

```
Add 5 and 7
```

Ma può avere difficoltà con:

```
Can you add five and seven?
```

Un LLM Parser può trasformare questa richiesta in:

```
{  
  "a": 5,  
  "b": 7  
}
```

Questo rende l'agente più flessibile con il linguaggio naturale.

Il Parser classico serve per imparare bene la logica.
Il Parser con LLM serve per gestire richieste più realistiche.

Multi-step Agent

Un agente avanzato deve poter eseguire più passaggi.

Esempio:

```
Add 2 and 3, then multiply the result by 4
```

L'agente deve costruire un piano come:

```
1. addition(2, 3)
2. multiplication(result, 4)
```

Questo introduce il concetto di orchestrazione multi-step.

L'agente non sceglie più solo un tool.
Costruisce una sequenza di azioni.

Questo è uno dei passaggi più importanti verso un agente moderno.

Roadmap del progetto

La roadmap completa è:

```
V1 → Agent base
V2 → Executor
V3 → Generic Parser
V4 → Planner
V5 → Memory Manager
V6 → Tool Registry
V7 → LLM Router
V8 → LLM Parser
V9 → Multi-step Agent
V10 → Professional Project Structure
```

Ogni versione ha un obiettivo preciso.

Il progetto cresce in modo ordinato:

```
prima funziona
poi si separa
poi diventa generico
poi pianifica
poi usa memoria gestita
poi registra tool
poi introduce LLM
poi diventa multi-step
poi diventa progetto professionale
```

Struttura professionale finale

Il notebook è perfetto per imparare, sperimentare e spiegare il progetto.

Alla fine, però, un agente professionale dovrebbe uscire dal notebook e diventare un progetto modulare.

Una possibile struttura finale è:

```
tools.py
router.py
parser.py
executor.py
memory.py
planner.py
agent.py
```

Ogni file contiene una responsabilità specifica.

Questa struttura rende il progetto:

- più leggibile;
- più testabile;
- più manutenibile;
- più facile da estendere;
- più vicino a un progetto reale.

Valore didattico

Il progetto **Building AI Agent** ha un valore didattico molto forte.

Non insegna solo a scrivere codice.

Insegna a pensare come progettare un sistema.

I concetti principali sono:

- separazione delle responsabilità;
- modularità;
- evoluzione incrementale;
- test progressivi;
- uso consapevole delle LLM;
- organizzazione del codice;
- passaggio da notebook a progetto reale.

Ogni componente nasce per risolvere un problema preciso.

L'Executor nasce perché l'Agent non deve eseguire tutto direttamente.
Il Generic Parser nasce perché un parser pieno di `if` non scala.
Il Planner nasce perché un agente deve poter ragionare prima di agire.
Il Memory Manager nasce perché la memoria deve essere gestita.
Il Tool Registry nasce perché i tool devono essere organizzati.
Le LLM entrano solo quando la struttura base è già solida.

Conclusione

Il progetto **Building AI Agent** è un percorso graduale per costruire un agente AI da zero fino a una struttura più professionale.

La sua forza è l'evoluzione controllata.

Ogni versione introduce un concetto nuovo e migliora l'architettura senza creare confusione.

Il risultato finale non è solo un agente che funziona, ma un sistema progettato con logica, chiarezza e responsabilità separate.

Questo è il passaggio fondamentale da un semplice script a un vero progetto da sviluppatore professionista.